Introdução à Orientação a Objetos

Aula 02 — Tipos primitivos, operadores, estruturas de controle; Classe, objeto, atributos e métodos

Programação Orientada a Objetos

Semana 2

Sumário

1. Introdução
2. Variáveis e tipos primitivos
3. Condicionais
4. Loopings
5. Orientação a objetos: classe e objeto
6. Métodos
7. Composição e referência a objetos
8. Encerramento

Revisão da aula anterior

- JDK / JRE / JVM, ciclo `.java` → `javac` → `.class` → `java`
- Convenções: `CamelCase`, arquivo = nome da classe, `;` obrigatório
- Primeiro programa rodando no terminal e no IntelliJ

Hoje

- Variáveis, tipos primitivos e *casting*
- Estruturas de controle: condicionais e *loopings*
- Nosso primeiro passo em Orientação a Objetos: classe, objeto, atributos e métodos
- Projeto de exemplo: uma pequena livraria (`Livro` e `Autor`)

Declarando variáveis

```
1 double livroJava8;
2 double livroTDD;
3
4 livroJava8 = 59.90;
5 livroTDD = 59.90;
```

- Toda variável em Java precisa de um **tipo**
- Pode-se declarar e atribuir em um único passo:

```
1 double livroJava8 = 59.90;
```

Somando e imprimindo

```java
public class CalculadoraDeEstoque {
    public static void main(String[] args) {
        double livroJava8 = 59.90;
        double livroTDD = 59.90;
        double soma = livroJava8 + livroTDD;
        System.out.println("O total em estoque e " + soma);
    }
}
```

Saída: `O total em estoque e 119.8`

Tipos primitivos

- Inteiros: `byte`, `short`, `int`, `long` — diferem no tamanho (bytes)
- Ponto flutuante: `float`, `double`
- Outros: `char`, `boolean`
- Hoje, raramente se usa `short`/`byte` só para economizar memória: `int` é o padrão
- `String` não é primitivo, mas já usamos: representa texto

Atribuição copia o valor

```java
int numero = 4;
int outroNumeroIgual = numero;
numero = numero + 5;

System.out.println(numero);           // 9
System.out.println(outroNumeroIgual); // 4
```

- Tipo primitivo: a variável guarda uma **cópia** do valor
- Mudar `numero` depois não afeta `outroNumeroIgual`

Casting

```java
int livroJava8 = 59.90; // erro de compilacao!
// Type mismatch: cannot convert from double to int
```

- Cabe valor **menor** em variável **maior** (`int` → `double`), não o contrário
- Para forçar a conversão, usamos *casting*:

```java
double livroJava8 = 59.90;
int numeroInteiro = (int) livroJava8; // 59, perde a parte decimal
```

if / else

```
1 if (soma < 150) {
2     System.out.println("Seu estoque esta muito baixo!");
3 } else {
4     System.out.println("Seu estoque esta bom");
5 }
```

- Expressão entre parênteses: uma **expressão booleana** (`true` ou `false`)
- Bloco do `if` executa quando `true`; bloco do `else`, quando `false`

Operadores relacionais

>	maior
<	menor
>=	maior ou igual
<=	menor ou igual
==	igual (comparação!)
!=	diferente

- Cuidado: = atribui, == compara

Encadeando condições

```
1 if (soma < 150) {
2     System.out.println("Seu estoque esta muito baixo!");
3 } else if (soma >= 2000) {
4     System.out.println("Seu estoque esta muito alto!");
5 } else {
6     System.out.println("Seu estoque esta bom");
7 }
```

Operador ternário

```
1 double valor = 0;
2 if (v1 > v2) {
3     valor = 100;
4 } else {
5     valor = 0;
6 }
```

Forma equivalente, mais enxuta:

```
1 double valor = v1 > v2 ? 100 : 0;
```

Operadores lógicos

&&	E (*and*)
||	OU (*or*)
!	negação

```
1 if (v1 > v2 && v2 < v3) {
2     // e maior que v1 e menor que v3
3 }
```

while

```
1 double soma = 0;
2 int contador = 0;
3
4 while (contador < 35) {
5     double valorDoLivro = 59.90;
6     soma = soma + valorDoLivro;
7     contador = contador + 1;
8 }
```

- Repete enquanto a condição for `true`
- Esquecer de atualizar a condição = **looping infinito**

Operadores de atribuição compostos

```
1 soma = soma + valorDoLivro;    // equivalente a:
2 soma += valorDoLivro;
3
4 contador = contador + 1;       // equivalente a:
5 contador += 1;                 // equivalente a:
6 contador++;
```

Também existem: -=, *=, /=, -

for

```
1 // inicializacao; condicao; atualizacao
2 for (int contador = 0; contador < 35; contador++) {
3     soma += 59.90;
4 }
```

- Junta inicialização, condição e atualização em uma linha
- Variável de controle é comumente chamada de `i` (*index*)

continue e break

```java
for (int i = 0; i <= 10; i++) {
    if (i == 7) { continue; }  // pula o 7, imprime 0-6 e 8-10
    System.out.println(i);
}
```

```java
for (int i = 0; i <= 10; i++) {
    if (i == 7) { break; }     // para no 7, imprime so 0-6
    System.out.println(i);
}
```

O problema do código procedural

- `CalculadoraDeEstoque` guarda só o **preço** do livro
- E o nome, a descrição, o ISBN? Ficam soltos, sem conexão
- Orientação a objetos: agrupar **estado** (atributos) e **comportamento** (métodos) em um objeto

Uma classe é um molde

```java
public class Livro {
    String nome;
    String descricao;
    double valor;
    String isbn;
}
```

- Campos dentro do escopo da classe: **atributos**
- A classe **não** guarda valores — ela é só o molde

Criando e populando um objeto

```
1 Livro livro = new Livro();
2 livro.nome = "Java 8 Pratico";
3 livro.descricao = "Novos recursos da linguagem";
4 livro.valor = 59.90;
5 livro.isbn = "978-85-66250-46-6";
6
7 System.out.println(livro.nome);
```

- `new` pede à JVM um novo objeto do tipo `Livro`
- `.` (ponto) acessa os atributos do objeto

Classe x Objeto

- **Classe**: molde, especificação do que um objeto tem e faz
- **Objeto**: instância criada a partir do molde, com seus próprios valores
- Uma livraria tem uma classe `Livro`, mas milhares de objetos `Livro`
- Cada objeto guarda seu próprio nome, descrição, valor, ISBN

Por que criar um método?

- Repetir `System.out.println` para cada atributo, de cada livro → código repetido
- 100 livros = 400 linhas parecidas; mudar um atributo = mexer em 100 lugares
- Solução: isolar o comportamento comum dentro da classe

Definindo um método

```java
public class Livro {
    String nome;
    String descricao;
    double valor;
    String isbn;

    void mostrarDetalhes() {
        System.out.println("Nome: " + nome);
        System.out.println("Descricao: " + descricao);
        System.out.println("Valor: " + valor);
        System.out.println("ISBN: " + isbn);
    }
}
```

Chamando o método

```
1 Livro livro = new Livro();
2 livro.nome = "Java 8 Pratico";
3 livro.valor = 59.90;
4
5 livro.mostrarDetalhes();
```

- `void`: o método não retorna nenhum valor
- Mudar o comportamento agora é **um único ponto** do código

Métodos com parâmetro

```java
public void aplicaDescontoDe(double porcentagem) {
    valor -= valor * porcentagem;
}
```

```java
livro.aplicaDescontoDe(0.1); // 10% de desconto
```

- Parâmetro: nome + tipo, assim como uma variável
- Evita espalhar a mesma lógica pelo código

Ambiguidade e o this

```java
public void aplicaDescontoDe(double valor) {
    valor -= valor * valor; // ambiguo! usa o parametro
}
```

```java
public void aplicaDescontoDe(double porcentagem) {
    this.valor -= this.valor * porcentagem;
}
```

- this refere-se explicitamente ao atributo da classe
- Boa prática mesmo quando não há ambiguidade

Métodos com retorno

```java
boolean temAutor() {
    return this.autor != null;
}
```

```java
if (livro.temAutor()) {
    System.out.println(livro.autor.nome);
}
```

- `return` devolve um valor do tipo declarado (aqui, `boolean`)
- Sem `void`: o método precisa de `return`

Uma classe pode ter outra como atributo

```java
public class Autor {
    String nome;
    String email;
    String cpf;
}

public class Livro {
    String nome;
    double valor;
    Autor autor;
}
```

Isso é **composição**: `Livro` tem um `Autor`

Associando objetos

```
1 Autor autor = new Autor();
2 autor.nome = "Rodrigo Turini";
3
4 Livro livro = new Livro();
5 livro.autor = autor;
```

- `livro.autor` agora aponta pro **mesmo** objeto `autor`

Objetos guardam referência, não cópia

```
1 Autor autor = new Autor();
2 autor.nome = "Rodrigo Turini";
3
4 Livro livro = new Livro();
5 livro.autor = autor;
6
7 livro.autor.nome = "Guilherme Silveira";
8
9 System.out.println(autor.nome); // Guilherme Silveira!
```

- Diferente de primitivo: variável de objeto guarda **referência** (endereço)
- Mudar por um caminho afeta o mesmo objeto visto por outro caminho

Comparando objetos com ==

```java
Autor autor = new Autor();
autor.nome = "Rodrigo Turini";

Autor autor2 = new Autor();
autor2.nome = "Rodrigo Turini";

if (autor == autor2) {
    System.out.println("Iguais!");
} else {
    System.out.println("Diferentes!"); // aqui!
}
```

== compara **referência** (endereço), não os valores dos atributos

Resumo da aula

- Tipos primitivos, atribuição por cópia, *casting*
- Condicionais (`if`/`else`, ternário) e *loopings* (`while`, `for`)
- Classe (molde) x objeto (instância), atributos e métodos
- Parâmetros, `this`, `return`
- Composição e referência a objetos

Próxima aula

Modificadores de acesso e encapsulamento